CSE 221: Algorithms

Chapter 2 — Time Complexity

Prepared by Ariyan Hossain [AYO]

1 Some simplified rules

Algebra of asymptotic notation

- If $f(n) = \mathcal{O}(g(n))$ and $g(n) = \mathcal{O}(h(n))$, then $f(n) = \mathcal{O}(h(n))$
- If $f(n) = \mathcal{O}(k \cdot g(n))$ for any $k > 0$, then $f(n) = \mathcal{O}(g(n))$
- If $f_1(n) = \mathcal{O}(g_1(n))$ and $f_2(n) = \mathcal{O}(g_2(n))$, then

$$f_1(n) + f_2(n) = \mathcal{O}(\max(g_1(n), g_2(n)))$$

- If $f_1(n) = \mathcal{O}(g_1(n))$ and $f_2(n) = \mathcal{O}(g_2(n))$, then

$$f_1(n) \cdot f_2(n) = \mathcal{O}(g_1(n) \cdot g_2(n))$$

- $\mathcal{O}(c_1) + \mathcal{O}(c_2) + \mathcal{O}(c_3) = \mathcal{O}(1) = \mathcal{O}(c_1 + c_2 + c_3) = \mathcal{O}(c_4) = c_4 \cdot \mathcal{O}(1) = \mathcal{O}(1)$
- $n \cdot \mathcal{O}(1) = \mathcal{O}(n)$
- $\mathcal{O}(m) + \mathcal{O}(n) \neq \mathcal{O}(m + n)$
- $\mathcal{O}(m) \cdot \mathcal{O}(n) = c_1 m \cdot c_2 n = (c_1 c_2)(mn) = \mathcal{O}(mn)$
- $\mathcal{O}(m) \cdot \mathcal{O}(n) \cdot \mathcal{O}(p) \cdot \mathcal{O}(q) = \mathcal{O}(mnpq)$
- $\mathcal{O}(an^2 + bn + c) = \mathcal{O}(n^2)$, where a, b, c are constants

2 Some series rules

Useful identities

Arithmetic Series:

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

$$S(n, p) = \sum_{i=1}^{n} i^p = \Theta(n^{p+1}) \quad (p \geq 0)$$

For $p \leq -1$, this sum converges to a constant.

Geometric Series:

$$2^0 + 2^1 + 2^2 + \cdots + 2^n = 2^{n+1} - 1$$

$$G(n, a) = \sum_{i=0}^{n} a^i = \frac{a^{n+1} - 1}{a - 1} = \Theta(a^{n+1}) \quad (a > 1)$$

For $a \leq 0$, this sum converges to a constant, and for $a = 1$, sum is n.

3 Iterative time complexities

3.1 Common patterns

Single loop with early break

```
for (j = 0; j < n; ++j)
    // 3 atomics
    if (condition) break;
```

- Upper bound: $\mathcal{O}(4n) = \mathcal{O}(n)$
- Lower bound: $\Omega(4) = \Omega(1)$
- Complexity: $\mathcal{O}(n)$

If/else: take the maximum branch

```
if (condition)
    i = 0;
else
    for (j = 0; j < n; j++)
        a[j] = j;
```

- Upper bound: $\mathcal{O}(1 + n) = \mathcal{O}(n)$
- Lower bound: $\Omega(2) = \Omega(1)$
- Complexity: $\mathcal{O}(n)$

Consecutive loops: add complexities

```
for (j = 0; j < n; ++j)
    // 3 atomics
for (j = 0; j < n; ++j)
    // 5 atomics
```

- Complexity: $\mathcal{O}(3n + 5n) = \mathcal{O}(n)$

Nested loops: multiply complexities

```
for (j = 0; j < n; ++j)
    // 2 atomics
    for (k = 0; k < n; ++k)
        // 3 atomics
```

- Complexity: $\mathcal{O}((2 + 3n)n) = \mathcal{O}(n^2)$

3.2 Nested independent loop

Different bounds and step size

Statement	# of iterations / cost
`sum = 0;`	1
`for (i = 10; i ≤ n; i++)`	$(n-10)/1$
`  for (j = 0; j ≤ n; j += 2)`	$(n-0)/2$
`  sum++;`	1
Total	$1 + ((n-10)/1) \cdot ((n-0)/2) \cdot 1 = \mathcal{O}(n^2)$

3.3 Nested dependent loops

Inner loop depends on outer loop

Statement	# of iterations / cost
`sum = 0;`	1
`for (i = 1; i ≤ n; i++)`	
`  for (j = 1; j ≤ i; j++)`	n^2 (Calculation shown below)
`  sum++;`	1
`for (i = 1; i ≤ n; i++)`	n
`  sum++;`	1
Total	$1 + n^2 \cdot 1 + n \cdot 1 = \mathcal{O}(n^2)$

A quick way to see the inner loop work is:

i	j	count
1	1	1
2	1, 2	2
3	1, 2, 3	3
...	...	...
n	$1, 2, 3, \ldots, n$	n

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2} = \mathcal{O}(n^2).$$

3.4 Nested independent loop with geometric growth

Outer loop doubles each step

Statement	**# of iterations / cost**
`sum = 0;`	1
`for (i = 1; i ≤ n; i*=2)`	$\log_2 n$ (Calculation shown below)
`for (j = 1; j ≤ n; j++)`	n
`sum++;`	1
Total	$1 + \log_2 n \cdot n \cdot 1 = \mathcal{O}(n \log_2 n)$

A quick way to see the inner loop work is:

Step	i
0	$1\,(2^0)$
1	$2\,(2^1)$
2	$4\,(2^2)$
$\cdots$	$\cdots$
k	$n\left(2^k\right)$

$$n = 2^k$$
$$\log_2 n = \log_2\left(2^k\right)$$
$$k = \log_2 n$$

3.5 Nested dependent loop with geometric growth

Outer loop doubles each step while inner loop depends on outer loop

Statement	# of iterations / cost
`sum = 0;`	1
`for (i = 1; i ≤ n; i*=2)`	
`  for (j = 1; j ≤ i; j++)`	n (Calculation shown below)
`    sum++;`	1
Total	$1 + n \cdot 1 = \mathcal{O}(n)$

A quick way to see the inner loop work is:

Step	i	j	count
0	$1\,(2^0)$	1	1
1	$2\,(2^1)$	$1, 2$	2
2	$4\,(2^2)$	$1, 2, 3, 4$	4
$\dots$	$\dots$	$\dots$	$\dots$
k	$n\left(2^k\right)$	$1, 2, 3, 4, \dots, n$	n

$$\begin{aligned} 1 + 2 + 4 + \cdots + n &= 2^0 + 2^1 + 2^2 + \cdots + 2^k \\ &= 2^{k+1} - 1 \\ &= 2 \cdot 2^k - 1 \\ &= 2n - 1 \\ &= \mathcal{O}(n). \end{aligned}$$

3.6 Quick Loop Patterns

Some common loop patterns

```
  for (i = 0; i <= n; i ++)   →  O(n)
for (i = 0; i <= n; i += 2)   →  O(n)
 for (i = n; i > 0; i -= 3)   →  O(n)

for (i = 1; i <= n; i *= 2)   →  O(log_2 n)
for (i = 1; i <= n; i *= 3)   →  O(log_3 n)
 for (i = n; i > 0; i /= 4)   →  O(log_4 n)
```

4 Recursive time complexities

With an iterative algorithm, we can analyze loops to get an accurate estimate of time complexity in most cases. However, this becomes much trickier if recursion is involved.

Recursive function

A recursive function has two parts: the **base case** and the **recursive case**. The recursive case is when the function calls itself. The base case is when the function does not go into an infinite loop).

Example of a recursive function

```
function position(n):
    if (n == 0):
        return 1
    else:
        return 1 + position(n-1)
```

Recursion is used when it makes the solution clearer. There is no performance benefit to using recursion; loops are sometimes better for performance.

4.1 Recurrences

Previously, we defined the runtime of an algorithm as a function $T(n)$ of its input size n. We still do this when dealing with recursive functions. However, because a recursive algorithm calls itself with a smaller input size, we write $T(n)$ as a **recurrence relation**, an equation that defines the runtime of a problem in terms of the runtime of a recursive call on smaller input.

A recurrence is an equation that describes a function in terms of its value on other (typically smaller) arguments. Recurrences go hand in hand with divide-and-conquer because they give a natural way to characterize running times mathematically.

An *algorithmic recurrence* is just a formula that tells you how much time your divide-and-conquer algorithm takes to solve a problem of size n.

4.2 Linear recurrence relations

A typical linear recurrence (e.g., for factorial) looks like:

$$T(n) = \begin{cases} \Theta(1), & \text{if } n = 0, \\ T(n-1) + \Theta(1), & \text{if } n > 0. \end{cases}$$

Here $T(n-1)$ is the runtime of the recursive call on a smaller input, and the extra $\Theta(1)$ is the constant work done outside the recursive call.

Converting a recursive function into a recurrence

```
function foo(n):
    if (n == 1):
        return 1
    else:
        return foo(n-1) + foo(n-1) + 1
```

This corresponds to:

$$T(n) = \begin{cases} \Theta(1), & \text{if } n = 1, \\ 2T(n-1) + \Theta(1), & \text{if } n > 1. \end{cases}$$

From here on, we can substitute $\Theta(1)$ with the constant 1 (this simplifies math without changing the asymptotic result):

$$T(n) = \begin{cases} 1, & \text{if } n = 1, \\ 2T(n-1) + 1, & \text{if } n > 1. \end{cases}$$

4.3 Ways to solve recurrences

- Master Theorem Method
- Substitution Method
- Recurrence Tree Method (optional)

4.3.1 Master Theorem Method

If the recurrence relation is of the form

$$T(n) = aT\left(\frac{n}{b}\right) + f(n),$$

where $a \geq 1$, $b > 1$, and $f(n)$ is an asymptotically positive function with complexity $\Theta(n^c)$, then:

Master Theorem Cases

$$T(n) = \begin{cases} \Theta\left(n^{\log_b(a)}\right), & \text{if } a > b^c, \\ \Theta(n^c \log_b(n)), & \text{if } a = b^c, \\ \Theta(n^c), & \text{if } a < b^c. \end{cases}$$

How to use it

1. Determine the values of a, b, and c.
2. Make sure the Master Theorem applies:
   - the coefficient a is at least 1,
   - the recursive argument divides by $b > 1$,
   - $f(n)$ is $\Theta(n^c)$ for some constant c,
   - a base case exists and costs constant time.
3. Compare a and b^c to choose the correct case.

Worked examples:

Example 1: $T(n) = 3T(n/2) + 5n + 13$

$$T(n) = \begin{cases} 1, & \text{if } n = 1, \\ 3T(n/2) + 5n + 13, & \text{if } n > 1. \end{cases}$$

Here $a = 3$, $b = 2$, and $f(n) = 5n + 13 = \mathcal{O}(n)$ so $c = 1$ and $b^c = 2$. Since $a > b^c$, we use Case 1:

$$T(n) = \mathcal{O}\Big(n^{\log_2 3}\Big) \approx \mathcal{O}\Big(n^{1.58}\Big).$$

Example 2: $T(n) = 6T(n/2) + 5n + 2^n$

$$T(n) = \begin{cases} 1, & \text{if } n = 1, \\ 6T(n/2) + 5n + 2^n, & \text{if } n > 1. \end{cases}$$

Here $f(n) = 5n + 2^n = \mathcal{O}(2^n)$, which is not of the form $\Theta(n^c)$, so the Master Theorem cannot be used directly.

Example 3: $T(n) = 7T(10n/11) + 9$

$$T(n) = \begin{cases} 1, & \text{if } n = 1, \\ 7T(10n/11) + 9, & \text{if } n > 1. \end{cases}$$

We can rewrite $10n/11 = n/(11/10)$, so $a = 7$, $b = \frac{11}{10}$, and $f(n) = 9 = \mathcal{O}(n^0)$ so $c = 0$ and $b^c = 1$. Since $a > b^c$, we use Case 1:

$$T(n) = \mathcal{O}\Big(n^{\log_{11/10} 7}\Big) \approx \mathcal{O}\Big(n^{20.4}\Big).$$

4.3.2 Substitution method

- **When to use:** Use the substitution method when the recurrence does not fit the Master Theorem form $aT(n/b) + f(n)$ cleanly, or when you want to derive an explicit closed form.
- **How to use:** Repeatedly expand $T(n)$ by substituting the recurrence definition (write $T(n)$ in terms of $T(n-1)$, then $T(n-2)$, etc.), identify the pattern in coefficients and added work, stop at the base case, and simplify the resulting sum (often arithmetic or geometric) to get the asymptotic bound.

Example 1: $T(n) = 2T(n-1) + 1$

$$T(n) = 2T(n-1) + 1, \qquad T(1) = 1.$$

Expand repeatedly and substitute:

$$\begin{aligned}
T(n) &= 2T(n-1) + 1 && \text{(Eq 1)} \\
T(n-1) &= 2T(n-2) + 1 && \text{(Putting this into Eq 1)} \\
T(n) &= 2\{2T(n-2) + 1\} + 1 \\
&= 4T(n-2) + 2 + 1 && \text{(Eq 2)} \\
T(n-2) &= 2T(n-3) + 1 && \text{(Putting this into Eq 2)} \\
T(n) &= 4\{2T(n-3) + 1\} + 2 + 1 \\
T(n) &= 8T(n-3) + 4 + 2 + 1 && \text{(Eq 3)} \\
&= 2^{n-1}T\big(n-(n-1)\big) + 2^{n-2} + 2^{n-3} + \cdots + 4 + 2 + 1 \\
&= 2^{n-1} + 2^{n-2} + 2^{n-3} + \cdots + 4 + 2 + 1 \\
&= 2^n - 1 \\
&= \mathcal{O}(2^n).
\end{aligned}$$

Example 2: $T(n) = T(n-1) + n$

$$T(n) = T(n-1) + n, \qquad T(1) = 1.$$

Expand repeatedly and substitute:

$$\begin{aligned}
T(n) &= T(n-1) + n && \text{(Eq 1)} \\
T(n-1) &= T(n-2) + (n-1) && \text{(Putting this into Eq 1)} \\
T(n) &= T(n-2) + (n-1) + n && \text{(Eq 2)} \\
T(n-2) &= T(n-3) + (n-2) && \text{(Putting this into Eq 2)} \\
T(n) &= T(n-3) + (n-2) + (n-1) + n && \text{(Eq 3)} \\
&= T(n-(n-1)) + (n-(n-1-1)) + (n-(n-1-1-1)) + \cdots + n \\
&= 1 + 2 + 3 + \cdots + n \\
&= \frac{n(n+1)}{2} \\
&= \mathcal{O}(n^2).
\end{aligned}$$